

CSCE 608 DATABASE SYSTEMS · MODULE 12 · LECTURE NOTES

Column Stores and Analytics

A different workload needs a different system.

MODULE OUTCOMES

- Contrast OLTP and OLAP workloads.
- Explain why column storage suits analytics.
- Explain the compression schemes columns enable.
- Explain late materialisation and vectorised execution together.
- Explain why the two workloads ended up in separate systems.

1 Two workloads

	OLTP	OLAP
Query shape	Few rows, all columns. 'Fetch order 12345.'	Many rows, few columns. 'Average order value by region last quarter.'
Writes	Constant and small — individual inserts and updates from users.	Bulk loads, infrequent updates. Often append-only.
Concurrency	Thousands of concurrent users.	A handful of large queries.
Latency target	Milliseconds.	Seconds to minutes.
Data size	Working set usually fits in memory.	Far larger than memory. Scans dominate.
Indexes	Many, and selective.	Few — a query touching 40% of rows gains nothing from an index (Module 03).

ROW STORAGE READS COLUMNS YOU DID NOT ASK FOR

Consider `SELECT avg(salary) FROM employees` on a table with fifty columns.

Row storage stores each row contiguously, and the unit of I/O is a page, so reading any row's salary requires reading all fifty of its columns.

You wanted four bytes per row and you read four hundred. Ninety-nine percent of the bandwidth — disk, memory, and cache — is spent on data the query does not reference.

Column storage keeps each column contiguously, so the query reads the salary column and nothing else. **That is a hundredfold reduction in bytes moved before any compression is applied**, and it is the first of the two reasons column stores win.

2 Compression

A COLUMN IS A FAR BETTER COMPRESSION TARGET THAN A ROW

A column contains values of a single type with a similar distribution — a million timestamps, or a million country codes, or a million prices in a narrow range.

A row contains a name, a date, a floating-point number, and a boolean, which have nothing in common and compress poorly together.

So column data is highly redundant, and the schemes that exploit it are simple, fast, and specialised.

Ratios of 10 to 30 times are routine, against perhaps 2 or 3 for row-oriented data with a general-purpose compressor.

And compression makes analytical queries faster rather than slower. Module 07 established that memory bandwidth, not CPU, is usually the limit for scans. Moving a tenth as many bytes and spending a few cycles decompressing is a clear win — and often the decompression is not needed at all, per the next page.

Scheme	Effective when	Direct operation
Run-length encoding	The column is sorted, or has low cardinality. Store (value, count) pairs.	Yes. Counting rows matching a value is summing run lengths — the data is never expanded.
Dictionary encoding	Few distinct values relative to row count. Store codes and a dictionary.	Yes. Filtering on 'Texas' means looking up its code once and comparing small integers rather than strings.
Bit-packing	Values span a small range. Store each in the minimum bits needed.	Yes. SIMD instructions operate on several packed values at once.
Delta encoding	Sorted numerics, timestamps, sequential identifiers. Store differences.	Partly — sums and ranges, yes; equality, no.
Frame of reference	Values clustered around a base. Store the base plus small offsets.	Yes, for comparisons against the base.

OPERATING ON COMPRESSED DATA DIRECTLY IS THE REAL WIN

Decompressing a column and then processing it captures only the I/O benefit. **Processing the compressed representation captures the CPU benefit too**, and that is where most of the advantage lies.

Run-length encoded: `SELECT count(*) WHERE state = 'TX'` is a sum over run lengths. The number of operations is proportional to the number of *runs*, not rows.

Dictionary encoded: resolve the predicate value to its code once, then scan comparing fixed-width integers — which is both faster per comparison than string comparison and amenable to SIMD.

So compression here is not a space-for-time trade at all. It saves space *and* time simultaneously, which is unusual enough to be worth noticing, and it is why analytical systems compress aggressively rather than treating it as an option.

3 Execution

	Early materialisation	Late materialisation
Approach	Reconstruct rows immediately after reading the needed columns, then process rows as usual.	Keep columns separate through filters, aggregations, and joins , assembling rows only at the very end.
Simplicity	Simple — the rest of the engine is unchanged.	More complex; operators work on column vectors.
Benefit retained	Only the I/O saving. The CPU advantages are discarded at reconstruction.	A filter on one column produces a bitmap or selection vector, and other columns are read only where it is set. Compression survives further into the plan, so more operators work on compressed data.

COLUMN STORAGE AND VECTORISED EXECUTION ARE THE SAME IDEA

Module 07 established that **vectorised execution wants dense arrays of same-typed values** to loop over, so that dispatch is amortised and the compiler can emit SIMD.

Column storage produces exactly that, by construction. A column is a dense array of same-typed values.

So the two ideas reinforce one another, and arriving at either tends to lead to the other. This is what happened historically: MonetDB started from column storage and arrived at vectorised execution; the X100 work made the connection explicit.

Together they account for most of the 10 to 100 times that analytical systems achieve over row stores on scan-heavy queries. Neither alone gets close, which is why systems that adopted only one saw disappointing results.

4 Why they remain separate systems

THE REQUIREMENTS ARE GENUINELY OPPOSED

OLTP wants row storage (a transaction touches whole rows), many selective indexes, fine-grained locking or MVCC, and short transactions with millisecond latency.

OLAP wants column storage, few indexes, bulk loading, heavy compression, long scans, and large memory allocations per query.

A system optimised for one is structurally poor at the other. This is not a maturity gap that will close with effort — the two sets of requirements conflict at the level of data layout, which is the most expensive thing to change.

Hence the standard architecture: an OLTP system of record, an ETL or change-data-capture pipeline (Module 11), and a separate analytical warehouse. **It is operationally annoying** — two systems, a pipeline, and a replication lag to explain to users — **which is why it keeps being questioned and keeps being rediscovered.**

- **HTAP systems** attempt both workloads in one engine, typically by maintaining a row store for recent writes and a column store for historical reads, with background conversion between them. SingleStore, TiDB, and SAP HANA take variations of this approach. It works, and the complexity is real.
- **Column indexes within row-store systems.** SQL Server's clustered columnstore index and PostgreSQL extensions such as Citus Columnar bring columnar storage to specific tables inside a conventional system.
- **DuckDB** made embedded analytics trivial: a column store with vectorised execution, in a library, with no server to operate. It has substantially changed how local analytical work is done, in the way SQLite did for transactional work.
- **Open formats decoupled storage from the engine.** **Parquet** (on-disk columnar) and **Arrow** (in-memory columnar) are read and written by many independent engines, so the data no longer belongs to one system. **This is arguably the most consequential recent change** — the format became the interface, and the engine became replaceable.
- **And the separation persists**, because the conflict in §4 has not gone away. The tooling around it has improved considerably; the underlying tension has not.

Where to see this taught differently

Original material only goes so far — hearing the same idea in another voice is how it sticks. These are free, and they cover this module specifically.

Resource	What it covers for this module
CMU 15-721 — Advanced Database Systems (free)	Pavlo's graduate course, largely about the systems in this module. The column store and compression lectures are the primary source.
Abadi, Boncz & Harizopoulos — The Design and Implementation of Modern Column-Oriented Database Systems (free)	The definitive survey. Late materialisation, compression, and the execution model, with measurements.

Resource	What it covers for this module
Stonebraker et al. — C-Store (2005, free)	The paper that established the modern column store, and the ancestor of Vertica.
Apache Arrow and Parquet documentation	The open formats of §4. Worth understanding directly — they are increasingly the interface between systems.
DuckDB documentation and papers	A readable, modern column store you can install in one command and experiment with immediately.

Exercises

Do these in code, not on paper, unless the exercise says otherwise. The module is not finished until they run.

- 1 Store the same dataset row-wise and column-wise. Measure bytes read for a single-column aggregate over all rows.
- 2 Measure the same query's wall-clock time for both layouts and attribute the difference.
- 3 Implement run-length and dictionary encoding for a column. Report the compression ratio on real data of each kind.
- 4 Implement a count query that operates on run-length encoded data without decompressing, and compare against decompress-then-count.
- 5 Implement dictionary-encoded filtering by comparing codes. Compare against string comparison.
- 6 Implement bit-packing and confirm the compiler vectorises operations on the packed representation.
- 7 Implement early and late materialisation for a two-predicate query and measure the difference.
- 8 Load the same data into PostgreSQL and DuckDB. Run five analytical queries on each and report the ratio.
- 9 Write a Parquet file and read it from two different tools, confirming the format rather than the engine is the interface.
- 10 Take an OLTP schema you know and design the analytical schema you would build from it, stating what the ETL would do.

Self-check

Answer these in writing, from memory, without looking back at the notes. If you cannot, you have read this module but not yet learned it — that is useful information, not a failure.

- 1 Give six differences between OLTP and OLAP workloads, and say which drives the rest.
- 2 Why does row storage waste bandwidth on an analytical query? Give the ratio.
- 3 Why does a column compress so much better than a row?

- 4 Why does compression make analytical queries faster rather than slower?
- 5 Name five column compression schemes and say which permit direct operation.
- 6 What is late materialisation and why does it win?
- 7 Why do column storage and vectorised execution belong together?
- 8 Why are OLTP and OLAP in separate systems, and is that a maturity gap?
- 9 Name four things that are changing, and say which is most consequential.